

Tournées efficaces bi-objectif obtenues par un couplage de deux véhicules autonomes

ROADEF'2021 :

22ème congrès annuel de la société Française de Recherche Opérationnelle et d'Aide à la Décision.
26-30 avril 2021 – UNIVERSITÉ DE HAUTE-ALSACE, MULHOUSE

Owein THUILLIER

et

Xavier GANDIBLEUX

FACULTÉ DES SCIENCES ET TECHNIQUES
UNIVERSITÉ DE NANTES
FRANCE

Organization of the Talk

1. Introduction
2. 1-objective TSP-D
3. 2-objective TSP-D
4. Summary

Introduction

The problem (1/2)

Context and resources:

- 1 depot and $n - 1$ customers to visit (delivery)
- 1 on-road vehicle (truck) and 1 unmanned air vehicle (drone)

1 drone mission:

- the drone leaves the truck, visits 1 customer, returns to truck

Common assumptions:

- The drone leaves/returns to the truck only when the truck is stationary at a customer location
- The drone or the truck may stationary wait the other at the rendez-vous location
- No constraint on (i) the drone flying distance (battery capacity is infinite), (ii) the drone transportation capacity

The problem (1/2)

Context and resources:

- 1 depot and $n - 1$ customers to visit (delivery)
- 1 on-road vehicle (truck) and 1 unmanned air vehicle (drone)

1 drone mission:

- the drone leaves the truck, visits 1 customer, returns to truck

Common assumptions:

- The drone leaves/returns to the truck only when the truck is stationary at a customer location
- The drone or the truck may stationary wait the other at the rendez-vous location
- No constraint on (i) the drone flying distance (battery capacity is infinite), (ii) the drone transportation capacity

The problem (1/2)

Context and resources:

- 1 depot and $n - 1$ customers to visit (delivery)
- 1 on-road vehicle (truck) and 1 unmanned air vehicle (drone)

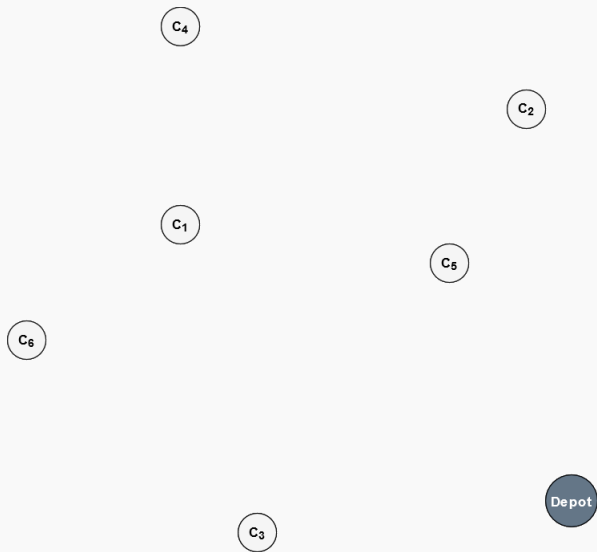
1 drone mission:

- the drone leaves the truck, visits 1 customer, returns to truck

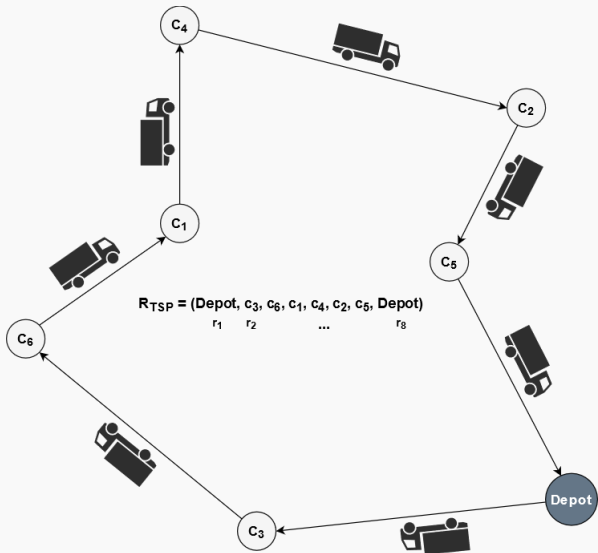
Common assumptions:

- The drone leaves/returns to the truck only when the truck is stationary at a customer location
- The drone or the truck may stationary wait the other at the rendez-vous location
- No constraint on (i) the drone flying distance (battery capacity is infinite), (ii) the drone transportation capacity

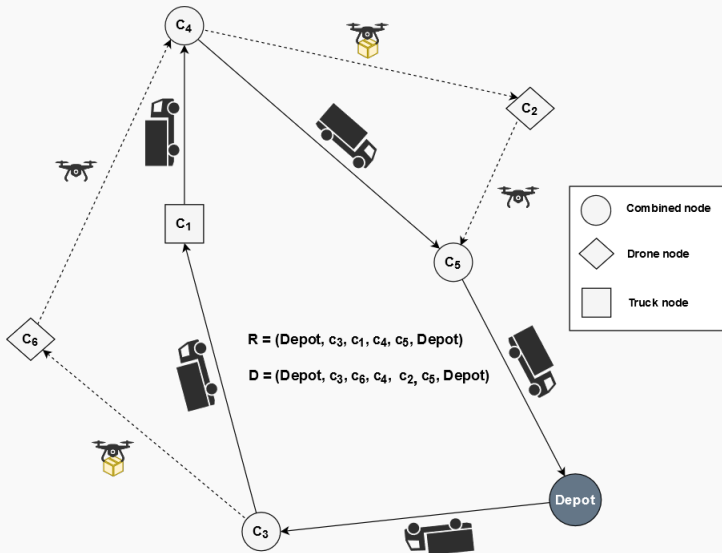
Example



Example



Example



The problem (2/2)

Questions:

- customers visited by both truck and drone ? → **combined nodes**
- customers visited by drone alone ? → **drone nodes**
- customers visited by truck alone ? → **truck nodes**

Objectives to optimize:

z_1 : minimize the **total travel time** [ABS18, PGW19]

z_2 : minimize the **total waiting time** (truck + drone) [HDPH18]

Goal is to compute:

- Y_N : the set of non-dominated points
- X_E : a complet set of efficient solutions

(see [Ehr05] for notations and definitions on multi-objective optimization)

The problem (2/2)

Questions:

- customers visited by both truck and drone ? → **combined nodes**
- customers visited by drone alone ? → **drone nodes**
- customers visited by truck alone ? → **truck nodes**

Objectives to optimize:

z_1 : minimize the **total travel time** [ABS18, PGW19]

z_2 : minimize the **total waiting time** (truck + drone) [HDPH18]

Goal is to compute:

- Y_N : the set of non-dominated points
- X_E : a complete set of efficient solutions

(see [Ehr05] for notations and definitions on multi-objective optimization)

The problem (2/2)

Questions:

- customers visited by both truck and drone ? → **combined nodes**
- customers visited by drone alone ? → **drone nodes**
- customers visited by truck alone ? → **truck nodes**

Objectives to optimize:

z_1 : minimize the **total travel time** [ABS18, PGW19]

z_2 : minimize the **total waiting time** (truck + drone) [HDPH18]

Goal is to compute:

- Y_N : the set of non-dominated points
- X_E : a complet set of efficient solutions

(see [Ehr05] for notations and definitions on multi-objective optimization)

- Murray-Chu [MC15]:
Beginning of stream of interest on TSP-D
- Agatz-Bouman-Schmidt [ABS18], algorithm A1:
a Dynamic programming algorithm optimizing objective z_1 under:
H1: The sequence for visiting customers is given from the optimal TSP R_{TSP} (truck alone in action) and frozen in the algorithm
H2: The truck cannot stay stationary when the drone leaves the truck
- Poikonen-Golden-Wasil [PGW19], algorithm A2:
extend the algorithm A1 where H3 replaces H2:
H3: The truck can stay stationary when the drone leaves the truck
- Ha-Deville-Pham-Hà [HDPH18]:
suggest the objective z_2 but z_1, z_2 are combined in a single objective
- Khoufi-Laouiti-Adjih [KLA19]:
recent survey, none exact algorithm for computing Y_N on z_1, z_2

Literature

- Murray-Chu [MC15]:
Beginning of stream of interest on TSP-D
- Agatz-Bouman-Schmidt [ABS18], algorithm A1:
a Dynamic programming algorithm optimizing objective z_1 under:
H1: The sequence for visiting customers is given from the optimal TSP R_{TSP} (truck alone in action) and frozen in the algorithm
H2: The truck cannot stay stationary when the drone leaves the truck
- Poikonen-Golden-Wasil [PGW19], algorithm A2:
extend the algorithm A1 where H3 replaces H2:
H3: The truck can stay stationary when the drone leaves the truck
- Ha-Deville-Pham-Hà [HDPH18]:
suggest the objective z_2 but z_1, z_2 are combined in a single objective
- Khoufi-Laouiti-Adjih [KLA19]:
recent survey, none exact algorithm for computing Y_N on z_1, z_2

Literature

- Murray-Chu [MC15]:
Beginning of stream of interest on TSP-D
- Agatz-Bouman-Schmidt [ABS18], algorithm A1:
a Dynamic programming algorithm optimizing objective z_1 under:
H1: The sequence for visiting customers is given from the optimal TSP R_{TSP} (truck alone in action) and frozen in the algorithm
H2: The truck cannot stay stationary when the drone leaves the truck
- Poikonen-Golden-Wasil [PGW19], algorithm A2:
extend the algorithm A1 where H3 replaces H2:
H3: The truck can stay stationary when the drone leaves the truck
- Ha-Deville-Pham-Hà [HDPH18]:
suggest the objective z_2 but z_1, z_2 are combined in a single objective
- Khoufi-Laouiti-Adjih [KLA19]:
recent survey, none exact algorithm for computing Y_N on z_1, z_2

Literature

- Murray-Chu [MC15]:
Beginning of stream of interest on TSP-D
- Agatz-Bouman-Schmidt [ABS18], algorithm A1:
a Dynamic programming algorithm optimizing objective z_1 under:
H1: The sequence for visiting customers is given from the optimal TSP R_{TSP} (truck alone in action) and frozen in the algorithm
H2: The truck cannot stay stationary when the drone leaves the truck
- Poikonen-Golden-Wasil [PGW19], algorithm A2:
extend the algorithm A1 where H3 replaces H2:
H3: The truck can stay stationary when the drone leaves the truck
- Ha-Deville-Pham-Hà [HDPH18]:
suggest the objective z_2 but z_1, z_2 are combined in a single objective
- Khoufi-Laouiti-Adjih [KLA19]:
recent survey, none exact algorithm for computing Y_N on z_1, z_2

Literature

- Murray-Chu [MC15]:
Beginning of stream of interest on TSP-D
- Agatz-Bouman-Schmidt [ABS18], algorithm A1:
a Dynamic programming algorithm optimizing objective z_1 under:
H1: The sequence for visiting customers is given from the optimal TSP R_{TSP} (truck alone in action) and frozen in the algorithm
H2: The truck cannot stay stationary when the drone leaves the truck
- Poikonen-Golden-Wasil [PGW19], algorithm A2:
extend the algorithm A1 where H3 replaces H2:
H3: The truck can stay stationary when the drone leaves the truck
- Ha-Deville-Pham-Hà [HDPH18]:
suggest the objective z_2 but z_1, z_2 are combined in a single objective
- Khoufi-Laouiti-Adjih [KLA19]:
recent survey, none exact algorithm for computing Y_N on z_1, z_2

1-objective TSP-D

Overview of algorithms A1 and A2

$R_{TSP} = (r_1 = Depot, \dots, r_{n+1} = Depot)$ (n : No. of customers + depot)

Notations:

- $(r_i, \dots, r_j)$: subsequence of R_{TSP}
($1 \leq i < j \leq n+1$)
- (i, j, k) : operation from r_i to r_j with drone node r_k (or -1 if
($1 \leq i < j \leq n+1$)
no drone node, i.e. $j = i + 1$)
- $V(i)$: minimum time to reach r_i (combined node)
- $P(i)$: previous combined node (index in R_{TSP})
- $T(i, j, k)$: travel time of the corresponding operation
- $T(i, j)$: travel time of best operation from r_i to r_j
- $M(i, j)$: drone node (index in R_{TSP}) of best operation from r_i to r_j

T and M are pre-computed and are the only elements differentiating A1 and A2

Overview of algorithms A1 and A2

$R_{TSP} = (r_1 = Depot, \dots, r_{n+1} = Depot)$ (n : No. of customers + depot)

Notations:

- $(r_i, \dots, r_j)$: subsequence of R_{TSP}
($1 \leq i < j \leq n+1$)
- (i, j, k) : operation from r_i to r_j with drone node r_k (or -1 if
($1 \leq i < j \leq n+1$)
no drone node, i.e. $j = i + 1$)
- $V(i)$: minimum time to reach r_i (combined node)
- $P(i)$: previous combined node (index in R_{TSP})
- $T(i, j, k)$: travel time of the corresponding operation
- $T(i, j)$: travel time of best operation from r_i to r_j
- $M(i, j)$: drone node (index in R_{TSP}) of best operation from r_i to r_j

T and M are pre-computed and are the only elements differentiating A1 and A2

Overview of algorithms A1 and A2

$R_{TSP} = (r_1 = Depot, \dots, r_{n+1} = Depot)$ (n : No. of customers + depot)

Notations:

- $(r_i, \dots, r_j)$: subsequence of R_{TSP}
($1 \leq i < j \leq n+1$)
- (i, j, k) : operation from r_i to r_j with drone node r_k (or -1 if
($1 \leq i < j \leq n+1$)
no drone node, i.e. $j = i + 1$)
- $V(i)$: minimum time to reach r_i (combined node)
- $P(i)$: previous combined node (index in R_{TSP})
- $T(i, j, k)$: travel time of the corresponding operation
- $T(i, j)$: travel time of best operation from r_i to r_j
- $M(i, j)$: drone node (index in R_{TSP}) of best operation from r_i to r_j

T and M are pre-computed and are the only elements differentiating A1 and A2

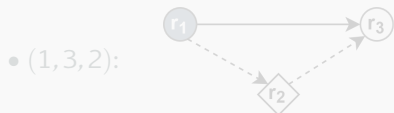
Overview of algorithms A1 and A2

Let $R_{TSP} = (r_1 = \text{Depot}, r_2, r_3, r_4, r_5 = \text{Depot})$ ($n = 4$)

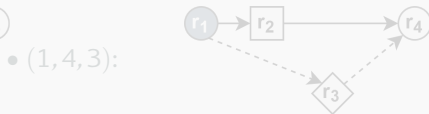
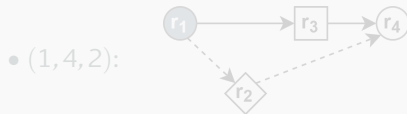
$$T(\overset{i}{1}, \overset{j}{2}) = T(\overset{i}{1}, \overset{j}{2}, \overset{k}{-1}); M(\overset{i}{1}, \overset{j}{2}) = \overset{k}{-1}$$



$$T(1, 3) = T(1, 3, 2); M(1, 3) = 2$$



$$T(1, 4) = \min\{T(1, 4, 2), T(1, 4, 3)\}; M(1, 4) \in \{2, 3\}$$



In A2, $k = -2$ and $k = -3$ are introduced for the stationary truck

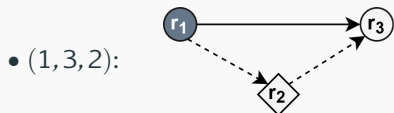
Overview of algorithms A1 and A2

Let $R_{TSP} = (r_1 = \text{Depot}, r_2, r_3, r_4, r_5 = \text{Depot})$ ($n = 4$)

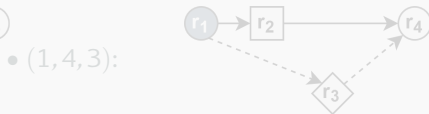
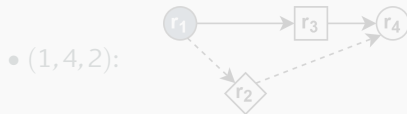
$$T(\overset{i}{1}, \overset{j}{2}) = T(\overset{i}{1}, \overset{j}{2}, \overset{k}{-1}); M(\overset{i}{1}, \overset{j}{2}) = \overset{k}{-1}$$



$$T(1, 3) = T(1, 3, 2); M(1, 3) = 2$$



$$T(1, 4) = \min\{T(1, 4, 2), T(1, 4, 3)\}; M(1, 4) \in \{2, 3\}$$



In A2, $k = -2$ and $k = -3$ are introduced for the stationary truck

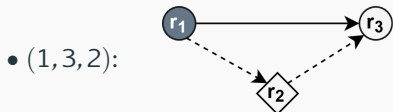
Overview of algorithms A1 and A2

Let $R_{TSP} = (r_1 = \text{Depot}, r_2, r_3, r_4, r_5 = \text{Depot})$ ($n = 4$)

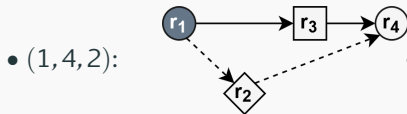
$$T(\overset{i}{1}, \overset{j}{2}) = T(\overset{i}{1}, \overset{j}{2}, \overset{k}{-1}); M(\overset{i}{1}, \overset{j}{2}) = \overset{k}{-1}$$



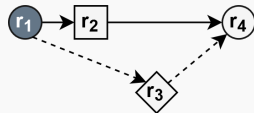
$$T(1, 3) = T(1, 3, 2); M(1, 3) = 2$$



$$T(1, 4) = \min\{T(1, 4, 2), T(1, 4, 3)\}; M(1, 4) \in \{2, 3\}$$



• $(1, 4, 3)$:



In A2, $k = -2$ and $k = -3$ are introduced for the stationary truck

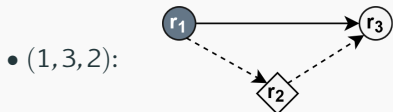
Overview of algorithms A1 and A2

Let $R_{TSP} = (r_1 = \text{Depot}, r_2, r_3, r_4, r_5 = \text{Depot})$ ($n = 4$)

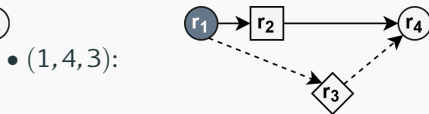
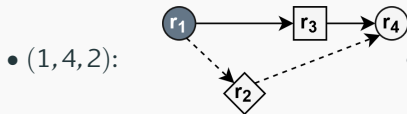
$$T(\overset{i}{1}, \overset{j}{2}) = T(\overset{i}{1}, \overset{j}{2}, \overset{k}{-1}); M(\overset{i}{1}, \overset{j}{2}) = \overset{k}{-1}$$



$$T(1, 3) = T(1, 3, 2); M(1, 3) = 2$$

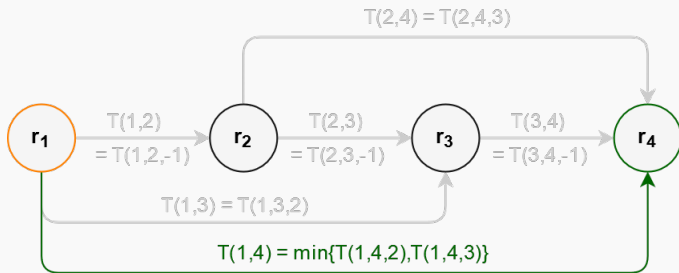


$$T(1, 4) = \min\{T(1, 4, 2), T(1, 4, 3)\}; M(1, 4) \in \{2, 3\}$$



In A2, $k = -2$ and $k = -3$ are introduced for the stationary truck

Overview of algorithms A1 and A2



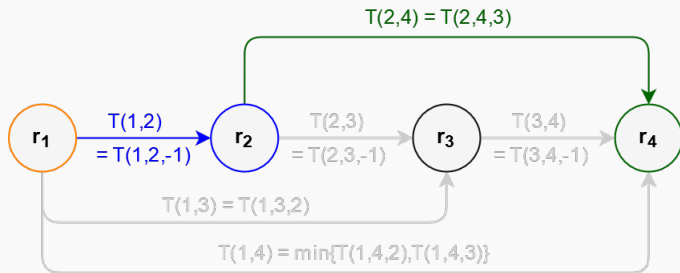
$$V(1) = 0; P(1) = -1$$

$$V(2) = V(1) + T(1,2); P(2) = 1$$

$$V(3) = \min\{V(1) + T(1,3), V(2) + T(2,3)\}; P(3) \in \{1, 2\}$$

$$V(4) = \min\{V(1) + T(1,4), V(2) + T(2,4), V(3) + T(3,4)\}; P(4) \in \{1, 2, 3\}$$

Overview of algorithms A1 and A2



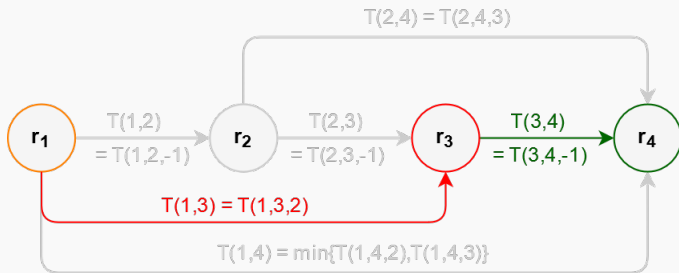
$$V(1) = 0; P(1) = -1$$

$$V(2) = V(1) + T(1,2); P(2) = 1$$

$$V(3) = \min\{V(1) + T(1,3), V(2) + T(2,3)\}; P(3) \in \{1, 2\}$$

$$V(4) = \min\{V(1) + T(1,4), V(2) + T(2,4), V(3) + T(3,4)\}; P(4) \in \{1, 2, 3\}$$

Overview of algorithms A1 and A2



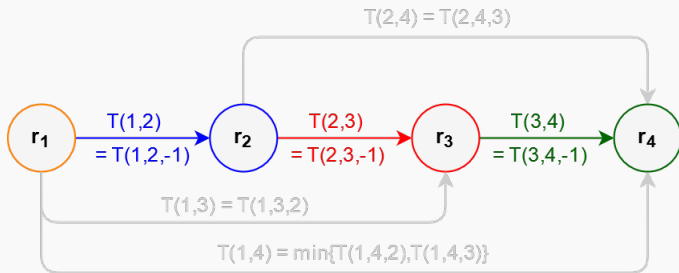
$$V(1) = 0; P(1) = -1$$

$$V(2) = V(1) + T(1,2); P(2) = 1$$

$$V(3) = \min\{V(1) + T(1,3), V(2) + T(2,3)\}; P(3) \in \{1, 2\}$$

$$V(4) = \min\{V(1) + T(1,4), V(2) + T(2,4), V(3) + T(3,4)\}; P(4) \in \{1, 2, 3\}$$

Overview of algorithms A1 and A2



$$V(1) = 0; P(1) = -1$$

$$V(2) = V(1) + T(1,2); P(2) = 1$$

$$V(3) = \min\{V(1) + T(1,3), V(2) + T(2,3)\}; P(3) \in \{1, 2\}$$

$$V(4) = \min\{V(1) + T(1,4), V(2) + T(2,4), V(3) + T(3,4)\}; P(4) \in \{1, 2, 3\}$$

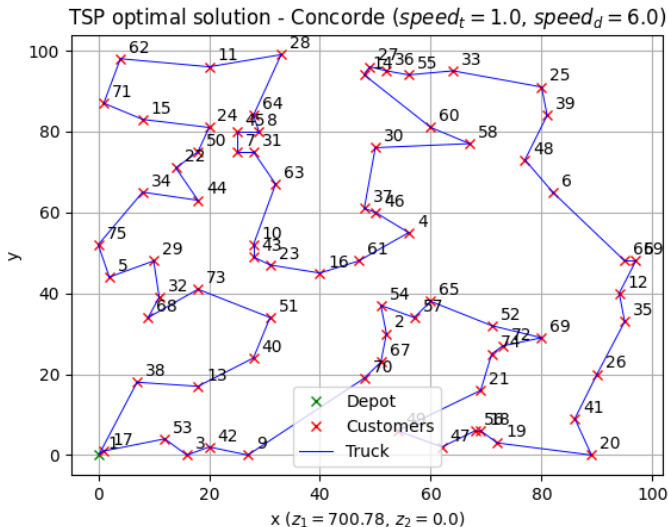
- Implemented in Julia the algorithms A1 and A2:
 - Concorde solver is called to compute the optimal solution
 - Linkern solver (Chained-Lin-Kernighan) is called to compute a heuristic solution
- Collected the datasets from the literature [ABS18, PGW19]
- Produced new instances (finer granularity)
- Made the parser compliant with the TSPLIB format

Production achieved

- Implemented in Julia the algorithms A1 and A2:
 - Concorde solver is called to compute the optimal solution
 - Linkern solver (Chained-Lin-Kernighan) is called to compute a heuristic solution
- Collected the datasets from the literature [ABS18, PGW19]
- Produced new instances (finer granularity)
- Made the parser compliant with the TSPLIB format

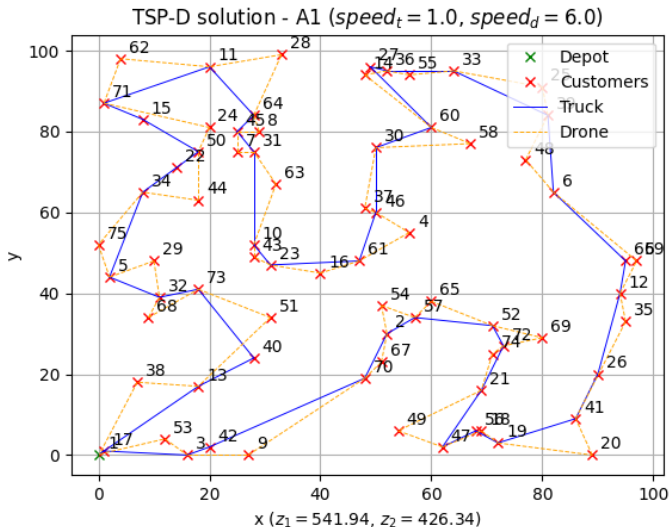
Example

"uniform-81-n75" from [ABS18]



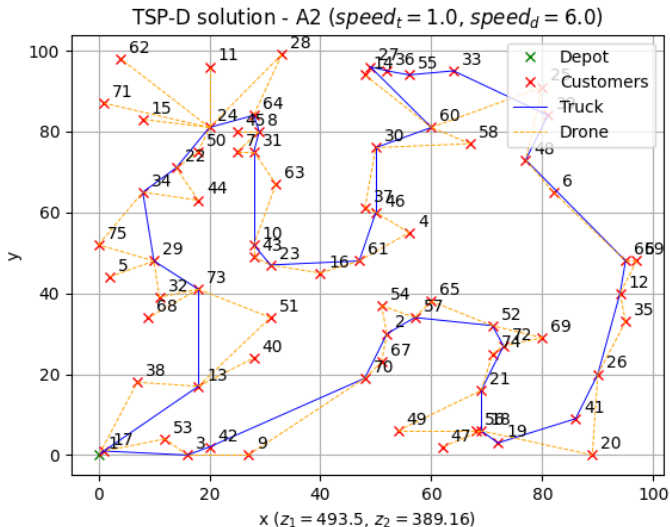
Example

"uniform-81-n75" from [ABS18]



Example

"uniform-81-n75" from [ABS18]



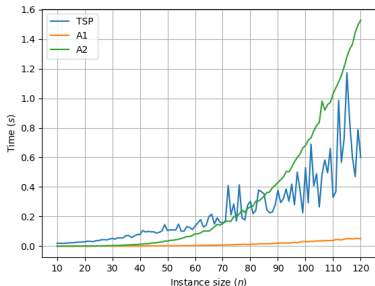
Experimentations

Environnement

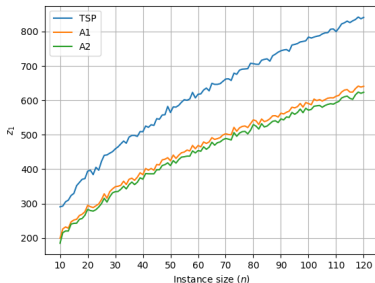
- OS: Linux (4.19.0-kali4-amd64)
- Manufacturer & product : HP Pavilion 15 Notebook PC
- CPU: Intel(R) Core(TM) i5-4210U CPU @ 1.70GHz
- Memory: 6GiB SODIMM DDR3 Synchrone Unbuffered (Unregistered) 1600 MHz (0,6 ns)
- Language: Julia (1.5.2)
- Solver: GLPK (4.60)
- Code verbosity: minimum
- Time limit: 60 seconds per instance

Experimentations

A1 vs A2: results with our instances



CPUt for several instance sizes



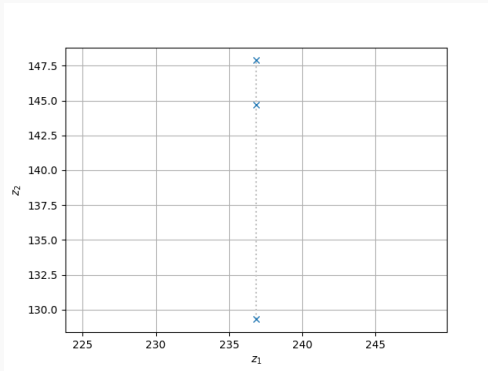
Value of z_1 for several instance sizes

- **Truck speed: 1; Drone speed: 3 ($\alpha = 3$)**
- **Size: $n \in \{10, \dots, 120\}$**
- For each size, 20 different instances were used (total : 2220)
- TSP solved optimally with Concorde
- **Variations in resolution times due to Concorde**

2-objective TSP-D

Observation 1

From an optimal solution given by A1 or A2, it is possible to improve the solution value on z_2 without degrading the optimal value observed on z_1 :

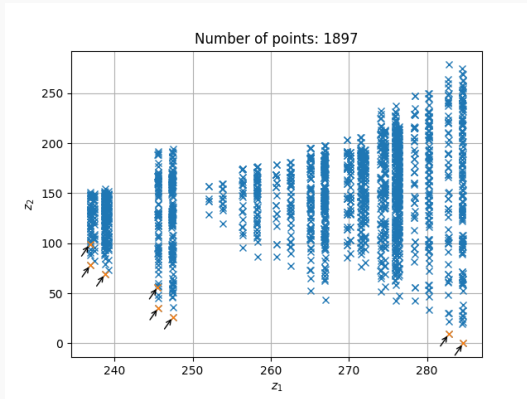


"uniform-53-n10" from [ABS18] with $\alpha = 3$

⇒ It exists weakly non-dominated points

Observation 2

From an exhaustive enumeration where solutions are filtered by dominance, several non-dominated points are noticed:

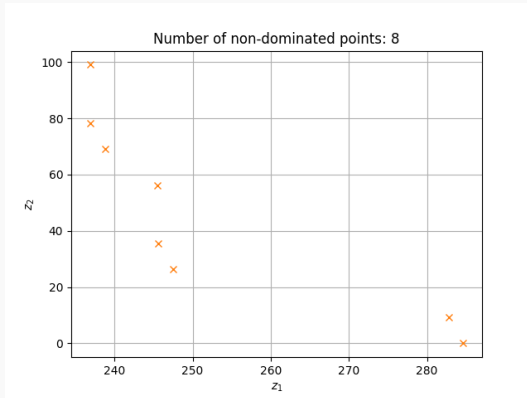


"uniform-53-n10" from [ABS18] with $\alpha = 3$

⇒ It exists non-dominated points

Observation 2

From an exhaustive enumeration where solutions are filtered by dominance, several non-dominated points are noticed:



"uniform-53-n10" from [ABS18] with $\alpha = 3$

⇒ It exists non-dominated points

Computing efficiently Y_N for z_1 and z_2

An implicit enumeration algorithm derivated from A1 and A2 (giving respectively 2A1 and 2A2) based on dynamic programming and filtering by dominance (**twice**).

Notations:

- $T(i, j, k)$: travel time of the corresponding operation
- $A(i, j, k)$: waiting time of the corresponding operation
- $M(i, j, k) = k$: drone node of the corresponding operation (or -1 if no drone node)
- $V_1[i][j]$: travel time to reach r_i (combined node) with j one possible path
- $V_2[i][j]$: waiting time to reach r_i (combined node) with j one possible path
- $P[i][j]$: previous combined node (relative to path j)

Computing efficiently Y_N for z_1 and z_2

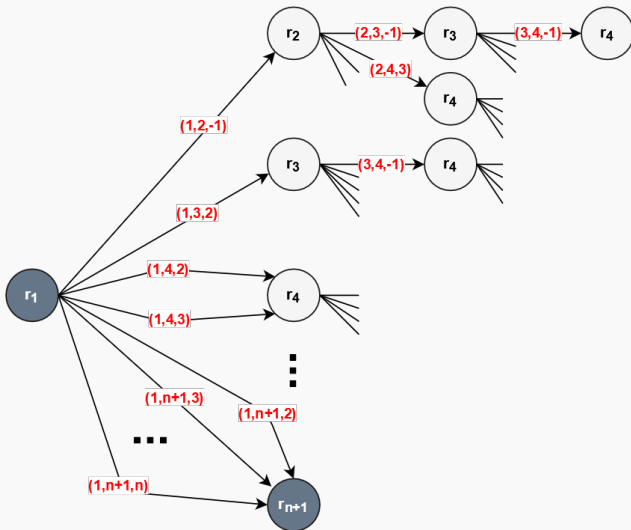
First combined node

Second ""

Third ""

Fourth ""

...



Computing efficiently Y_N for z_1 and z_2

Algorithm - Generation w/ iterative dominance filtering:

```
 $V_1[1] \leftarrow [0]; V_2[1] \leftarrow [0]; P[1] \leftarrow [[-1, -1, -1]]$   
for  $i \leftarrow 2$  to  $n + 1$  do  
  for  $k \leftarrow 1$  to  $i - 1$  do  
    for  $y \leftarrow 1$  to  $\text{length}(V_1[k])$  do  
      for  $z \leftarrow 1$  to  $\text{length}(T[k, i])$  do  
         $\text{append}(V_1[i], V_1[k][y] + T[k, i][z])$   
         $\text{append}(V_2[i], V_2[k][y] + A[k, i][z])$   
         $\text{append}(P[i], [k, y, M[k, i][z]])$   
      end for  
    end for  
  end for  
   $\text{filterByDominance}(V_1[i], V_2[i], P[i])$   
end for
```

Reconstruction of optimal path by backtracking from r_{n+1} with P

Computing efficiently Y_N for z_1 and z_2

Algorithm - Generation w/ iterative dominance filtering:

```
 $V_1[1] \leftarrow [0]; V_2[1] \leftarrow [0]; P[1] \leftarrow [[-1, -1, -1]]$   
for  $i \leftarrow 2$  to  $n + 1$  do  
  for  $k \leftarrow 1$  to  $i - 1$  do  
    for  $y \leftarrow 1$  to  $\text{length}(V_1[k])$  do  
      for  $z \leftarrow 1$  to  $\text{length}(T[k, i])$  do  
         $\text{append}(V_1[i], V_1[k][y] + T[k, i][z])$   
         $\text{append}(V_2[i], V_2[k][y] + A[k, i][z])$   
         $\text{append}(P[i], [k, y, M[k, i][z]])$   
      end for  
    end for  
  end for  
   $\text{filterByDominance}(V_1[i], V_2[i], P[i])$   
end for
```

z_1 z_2

Reconstruction of optimal path by backtracking from r_{n+1} with P

Experimentations

Results

n	$\#Y_N$	$\#Y_{SN}$	$\#Y_{NN}$	TIME(s)
10	9.6	5.5	4.1	0.002405
20	24.0	10.0	14.0	0.060915
30	51.2	14.5	36.7	0.635835
40	79.7	19.25	60.45	4.344919
50	118.95	24.0	94.95	15.608601
60	163.95	28.7	135.25	55.086532
70	-	-	-	> 60

$$X_{E_m} \cong X_E \cong X_{E_M}$$

- **Truck speed: 1; Drone speed: 3** ($\alpha = 3$)
- 20 different instances were used in each experimentation
- TSP solved optimally with Concorde (time not included here)

Summary

Conclusion and ongoing works

- Fixed errors in the original papers on the computations of $T(i,j)$
- Reproduced algorithms A1 and A2 in a single environment
- Performed a large number of experimentations on A1 and A2
- Introduced a bi-objective algorithm for the TSP-D (2A1/2A2)
- Reported numerical experiments:
 - Solving instances up to 60 customers within 60s
 - Observed a reasonable number of non-dominated points

Conclusion and ongoing works

Papers are mushrooming very fast!

Now:

- Comparison with a MIP model

e.g.:

Roberto Roberti and Mario Ruthmair. Exact Methods for the Traveling Salesman Problem with Drone. *Transportation Science* (accepted), 2020.

- Comparison with others DP approaches

e.g.:

Paul Bouman, Niels Agatz, Marie Schmidt. Dynamic programming approaches for the traveling salesman problem with drone. *Networks*. 2018; 72: 528– 542.

- Integrating others cuts into the bi-objective algorithm
- Integrating more constraints (battery, capacity)

-  Niels Agatz, Paul Bouman, and Marie Schmidt, *Optimization approaches for the traveling salesman problem with drone*, *Transportation Science* **52** (2018), no. 4, 965–981.
-  Matthias Ehrgott, *Multicriteria optimization*, 2 ed., Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2005.
-  Quang Minh Ha, Yves Deville, Quang Dung Pham, and Minh Hoàng Hà, *On the min-cost traveling salesman problem with drone*, *Transportation Research Part C: Emerging Technologies* **86** (2018), 597 – 621.
-  Ines Khoufi, Anis Laouiti, and Cedric Adjih, *A survey of recent extended variants of the traveling salesman and vehicle routing problems for unmanned aerial vehicles*, *Drones* **3** (2019), 66.

-  Chase C. Murray and Amanda G. Chu, *The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery*, Transportation Research Part C: Emerging Technologies **54** (2015), 86 – 109.
-  Sophie N. Parragh, Karl F. Doerner, and Richard F. Hartl, *A survey on pickup and delivery problems*, Journal für Betriebswirtschaft **58** (2008), no. 1, 21–51.
-  ———, *A survey on pickup and delivery problems*, Journal für Betriebswirtschaft **58** (2008), no. 2, 81–117.
-  Stefan Poikonen, Bruce Golden, and Edward A. Wasil, *A branch-and-bound approach to the traveling salesman problem with a drone*, INFORMS Journal on Computing **31** (2019), no. 2, 335–346.



Owein Thuillier and Théo Le Colleter and Xavier Gandibleux, *Bi-Objective Traveling Salesman Problem with Drone (TSP-D)*, International Workshop Recent Advances in Multi-Objective Optimization, Johannes Kepler University Linz, Austria. 17 September 2020.



Owein Thuillier and Théo Le Colleter, *Etude et contributions à la résolution du TSP-D*, Travail d'étude et de recherche (master 1 informatique, parcours "optimisation en recherche opérationnelle"), Université de Nantes, 2020, Défendu en Mai 2020. 223 pages.

Tournées efficaces bi-objectif
obtenues par un couplage de deux véhicules
autonomes

Owein THUILLIER
Xavier GANDIBLEUX